

PostgreSQL - Apunte

POSTGRESS - APUNTE

Postgres es la piedra fundacional de las bases de datos. Es una herramienta para gestionar bases de datos relacionales.

DAY 1: RELATIONS, CRUD Y JOINS

RELATIONS

Los prompts en postgres con el nombre de la base de datos, seguido de un símbolo hash (#) para los administradores y con un símbolo dólar (\$) para los usuarios comunes.

Postgres llama a las relaciones TABLES, a los atributos COLUMNS y a las tuplas ROWs.

Crear una tabla involucra darle a la tabla un nombre y una lista de columnas con tipos y (posiblemente) información acerca de restricciones.

```
CREATE TABLE countries (  
country_code char(2) PRIMARY KEY,  
country_name text UNIQUE  
);
```

Las restricciones pueden ser de tipo primary key, foreign key, si un valor puede ser null, etc...

En resumen, CRUD es Create, Read, Update, Delete. Create está más arriba.

Read:

```
SELECT *  
FROM countries;
```

Update:

```
UPDATE cities  
SET postal_code = '97206'  
WHERE name = 'Portland';
```

```
INSERT INTO cities
```

```
VALUES ('Toronto','M4C1B5','ca');
```

Delete:

```
DELETE FROM countries
WHERE country_code = '11';
```

JOIN READS

Los join son operaciones que toma 2 tablas separadas y las combina de algún modo para retornar una tabla más grande.

La forma más básica del join es el inner join:

```
SELECT cities.*, country_name
FROM cities INNER JOIN countries /* or just FROM cities
JOIN countries */
ON cities.country_code = countries.country_code;
```

También existen joins en tablas con claves primarias compuestas:

```
SELECT v.venue_id, v.name, c.name
FROM venues v INNER JOIN cities c
ON v.postal_code=c.postal_code AND
v.country_code=c.country_code;
```

THE OUTER LIMITS

Los outer joins son una forma de unir dos tablas cuando necesitamos que los resultados de una de las tablas siempre sea retornado, sin importar si hay o no valores en las columnas coincidentes en la otra tabla. A continuación, un ejemplo que muestra la diferencia:

```
SELECT e.title, v.name
FROM events e JOIN venues v
ON e.venue_id = v.venue_id;
```

title	name
Fight Club	Voodoo Doughnut

```
SELECT e.title, v.name
FROM events e LEFT JOIN venues v
```

```
ON e.venue_id = v.venue_id;
```

title		name
Fight Club		Voodoo Doughnut
April Fools Day		
Christmas Day		

FAST LOOKUPS WITH INDEXING

Se pueden usar índices para reducir los accesos a disco y mejorar la eficiencia de la base de datos:

```
CREATE TABLE / PRIMARY KEY will create implicit index  
"events_pkey" \  
for table "events"
```

Se usa por defecto B-tree, pero se puede explicitar el uso de otras estructuras (como una hashtable o un map):

```
CREATE INDEX events_title  
ON events USING hash (title);
```

DAY 2: ADVANCED QUERIES, CODE, AND RULES

En este día vamos a enfocarnos en formas de agrupar valores similares, ejecutar código en el servidor y crear interfaces personalizadas usando vistas y reglas.

AGGREGATE FUNCTIONS

Una query de agregación, agrupa resultados de varias filas bajo un criterio en común:

```
SELECT count(title)  
FROM events  
WHERE title LIKE '%Day%';
```

```
-----  
  
SELECT min(starts), max(ends)  
FROM events INNER JOIN venues  
ON events.venue_id = venues.venue_id  
WHERE venues.name = 'Crystal Ballroom';
```

GROUPING

Con la cláusula GROUP BY le decimos a Postgres que posicione las filas en grupos y luego realice una función de agregación:

```
SELECT venue_id, count(*)  
FROM events  
GROUP BY venue_id;
```

Para filtrar estos resultados usamos la cláusula HAVING:

```
SELECT venue_id  
FROM events  
GROUP BY venue_id  
HAVING count(*) >= 2 AND venue_id IS NOT NULL;
```

WINDOW FUNCTIONS

Permiten usar funciones de agregación a través de múltiples filas. La diferencia es que permiten usar funciones de agregación predefinidas sin que cada campo se agrupe en una sola fila:

```
SELECT title, count(*) OVER (PARTITION BY venue_id) FROM  
events
```

Se usan cuando hay campos “solapados”.

TRANSACTIONS

Las transacciones son un conjunto de queries que se ejecutan dentro de un bloque que siguen los principios ACID:

- Atomic (todo o nada)
- Consistent (la información siempre va a estar en buen estado, nunca en un estado inconsistente).
- Isolated (las transacciones no interfieren entre sí)
- Durable (una transacción exitosa está asegurada, incluso ante un fallo en el servidor)

Se marca el comienzo de una transacción con BEGIN TRANSACTION y el final con ROLLBACK o END (dependiendo si la transacción es exitosa o no:

```
BEGIN TRANSACTION;  
DELETE FROM events;  
ROLLBACK;  
SELECT * FROM events;
```

```
BEGIN TRANSACTION;  
UPDATE account SET total=total+5000.0 WHERE  
account_id=1337;  
UPDATE account SET total=total-5000.0 WHERE  
account_id=45887;  
END;
```

STORED PROCEDURES

Hay veces que la lógica de las queries no es suficiente para lograr los resultados esperados, por ende podemos delegar esta tarea a la aplicación o a procedimientos almacenados en Postgres.

Los procedimientos almacenados pueden ofrecer enormes ventajas de rendimiento a costa de un costo proporcional arquitectónico. Se le puede llamar a los procedimientos `FUNCTIONs`

postgres/add_event.sql

```
CREATE OR REPLACE FUNCTION add_event(  
    title text,  
    starts timestamp,  
    ends timestamp,  
    venue text,  
    postal varchar(9),  
    country char(2))  
RETURNS boolean AS $$  
DECLARE  
    did_insert boolean := false;  
    found_count integer;  
    the_venue_id integer;  
BEGIN  
    SELECT venue_id INTO the_venue_id  
    FROM venues v  
    WHERE v.postal_code=postal AND v.country_code=country AND v.name ILIKE venue  
    LIMIT 1;  
  
    IF the_venue_id IS NULL THEN  
        INSERT INTO venues (name, postal_code, country_code)  
        VALUES (venue, postal, country)  
        RETURNING venue_id INTO the_venue_id;  
  
        did_insert := true;  
    END IF;  
  
    -- Note: this is a notice, not an error as in some programming languages  
    RAISE NOTICE 'Venue found %', the_venue_id;  
  
    INSERT INTO events (title, starts, ends, venue_id)  
    VALUES (title, starts, ends, the_venue_id);  
  
    RETURN did_insert;  
END;  
$$ LANGUAGE plpgsql;
```

PULL THE TRIGGERS

Los triggers (gatillos o detonantes) automáticamente disparan una función ante un evento determinado (una inserción o una eliminación):

```
CREATE TRIGGER log_events  
AFTER UPDATE ON events  
FOR EACH ROW EXECUTE PROCEDURE log_event();
```

La función log_event() es:

postgres/log_event.sql

```
CREATE OR REPLACE FUNCTION log_event() RETURNS trigger AS $$
DECLARE
BEGIN
    INSERT INTO logs (event_id, old_title, old_starts, old_ends)
    VALUES (OLD.event_id, OLD.title, OLD.starts, OLD.ends);
    RAISE NOTICE 'Someone just changed event #%', OLD.event_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Finally, we create our trigger to log changes after any row is updated.

```
CREATE TRIGGER log_events
AFTER UPDATE ON events
FOR EACH ROW EXECUTE PROCEDURE log_event();
```

So, it turns out our party at Run's House has to end earlier than we hoped. Let's change the event.

```
UPDATE events
SET ends='2018-05-04 01:00:00'
WHERE title='House Party';
```

NOTICE: Someone just **changed event** #9

And the old end time was logged.

```
SELECT event_id, old_title, old_ends, logged_at
FROM logs;
```

VIEWING THE WORLD

Se usan para usar los resultados de una query compleja como otra tabla. Tiene dos aplicaciones principales:

- Para ocultar información sensible a usuarios no autorizados
- Para simplificar la sintaxis y el display de información

```
CREATE VIEW holidays AS
SELECT event_id AS holiday_id, title AS name, starts AS
date
FROM events
WHERE title LIKE '%Day%' AND venue_id IS NULL;
```

Para usar las vistas, se usan como cualquier otra tabla:


```
SELECT name, to_char(date, 'Month DD, YYYY') AS date
FROM holidays
WHERE date <= '2018-04-01';
```

I'LL MEET YOU AT THE CROSSTAB

Las tablas pivote “pivotatean” data agrupada alrededor de otro output (en el ejemplo, una lista de meses). Para hacerlo usamos la función crosstab(). Sea el ejemplo:

```
SELECT extract(year from starts) as year,
extract(month from starts) as month, count(*)
FROM events
GROUP BY year, month
ORDER BY year, month;
```

Creemos una tabla temporal:

```
CREATE TEMPORARY TABLE month_count(month INT);
INSERT INTO month_count VALUES (1),(2),(3),(4),(5),
(6),(7),(8),(9),(10),(11),(12);
```

Luego, finalmente:

```
SELECT * FROM crosstab(
'SELECT extract(year from starts) as year,
extract(month from starts) as month, count(*)
FROM events
GROUP BY year, month
ORDER BY year, month',
'SELECT * FROM month_count'
) AS (
year int,
jan int, feb int, mar int, apr int, may int, jun int,
jul int, aug int, sep int, oct int, nov int, dec int
) ORDER BY YEAR;
```

Esta última parte es relevante ya que provee información de cómo etiquetar las filas.

DAY 3: FULL TEXT AND MULTIDIMENSIONS

PostgreSQL - Tarea

Lista de comandos

ENTRAR A PGADMIN:

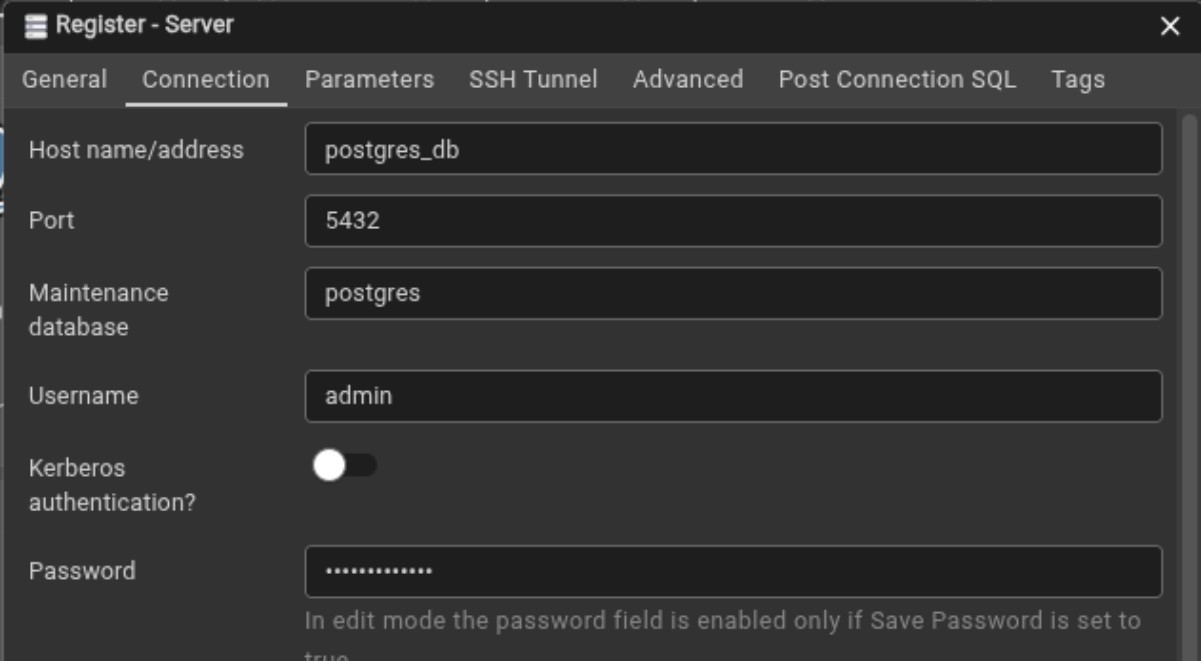
en el buscador:

localhost:8080

en la pantalla de inicio, poner esta información del dockercompose

```
pgadmin:
  image: dpage/pgadmin4
  container_name: pgadmin_ui
  restart: always
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@admin.com
    PGADMIN_DEFAULT_PASSWORD: admin
  ports:
    - "8080:80"
  depends_on:
    - db
```

En “add new server”, completar con la imagen de POSTGRES

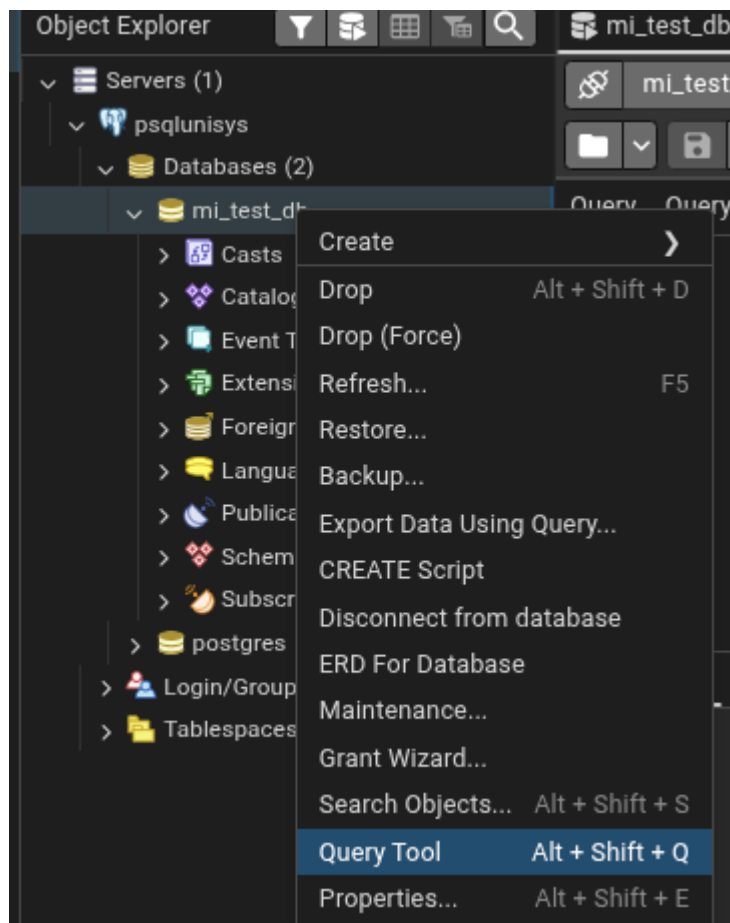


The screenshot shows the 'Register - Server' dialog box in pgAdmin. The 'Connection' tab is selected, and the following fields are filled out:

- Host name/address: postgres_db
- Port: 5432
- Maintenance database: postgres
- Username: admin
- Kerberos authentication: ☐
- Password: (masked with dots)

At the bottom, there is a note: "In edit mode the password field is enabled only if Save Password is set to true".

```
db:
  image: postgres:latest
  container_name: postgres_db ## Asi se llama el contenedor
  restart: always
  environment:
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: mipassword123
    POSTGRES_DB: mi_test_db
```



Y ahí aparece el espacio para ejecutar queries.

MOSTRAR DESPUÉS DE PRESENTAR EL CRUD

```
1 MOSTRAR CREATE
CREATE TABLE countries (
country_code char(2) PRIMARY KEY,
country_name text UNIQUE
);
```

Postgres posee, entre las bases de datos del libro, la mayor diversidad de datatypes.

```
2 MOSTRAR INSERT
INSERT INTO countries (country_code, country_name)
VALUES ('us','United States'), ('mx','Mexico'),
('au','Australia'),
('gb','United Kingdom'), ('de','Germany'),
('ll','Loompaland');
```

```
2.1 mostrar insercion arg
INSERT INTO countries (country_code, country_name)
VALUES ('AR', 'Argentina');
```

```
3 MOSTRAR ERROR DE RESTRICCIÓN DE CLAVE PRIMARIA
INSERT INTO countries
VALUES ('uk','United Kingdom');
```

```
4 MOSTRAR SELECT
SELECT *
FROM countries;
```

```
5 MOSTRAR DELETE
DELETE FROM countries
WHERE country_code = 'll';
```

```
6 CREATE TABLE cities (
name text NOT NULL,
postal_code varchar(9) CHECK (postal_code <> ''),
country_code char(2) REFERENCES countries,
PRIMARY KEY (country_code, postal_code)
);
```

```
7 MOSTRAR ERROR DE INTEGRIDAD DE CLAVE FORÁNEA (no existe
'ca' en las claves primarias de country)
INSERT INTO cities
```

```
VALUES ('Toronto','M4C1B5','ca');
```

```
8 INSERT INTO cities  
VALUES ('Portland','87200','us');
```

tiene que mostrar 0 1 , que es un insert.

```
9 MOSTRAR UPDATE  
UPDATE cities  
SET postal_code = '97206'  
WHERE name = 'Portland';
```

-----create read update y delete completado-----

```
10 JOIN  
SELECT cities.*, country_name  
FROM cities INNER JOIN countries  
ON cities.country_code = countries.country_code;
```

esto devuelve una tabla con todas las columnas de la tabla cities además del valor coincidente de country_name, de la tabla countries

```
11  
CREATE TABLE venues (  
venue_id SERIAL PRIMARY KEY,  
name varchar(255),  
street_address text,  
type char(7) CHECK ( type in ('public','private') )  
DEFAULT 'public',  
postal_code varchar(9),  
country_code char(2),  
FOREIGN KEY (country_code, postal_code)  
REFERENCES cities (country_code, postal_code) MATCH FULL  
);
```

venue_id es incremental
MATCH FULL obliga a que ambas claves foraneas existan o ambas sean nulas.

```
12  
INSERT INTO venues (name, postal_code, country_code)  
VALUES ('Crystal Ballroom', '97206', 'us');
```

13

```
SELECT v.venue_id, v.name, c.name
FROM venues v INNER JOIN cities c
ON v.postal_code=c.postal_code AND
v.country_code=c.country_code;
esto va a devolver la venue que insertamos recién.
```

14 CREAR TABLA EVENTS PARA HOMEWORK

```
CREATE TABLE events (
    event_id SERIAL PRIMARY KEY,
    title VARCHAR(255),
    starts TIMESTAMP,
    ends TIMESTAMP,
    venue_id INTEGER,
    FOREIGN KEY (venue_id) REFERENCES venues(venue_id)
);
```

15 INSERTS DE EVENTOS

Insert 1

```
INSERT INTO events (title, starts, ends, venue_id,
event_id)
VALUES ('Fight Club', '2018-02-15 17:30:00', '2018-02-15
19:30:00', 2, 1);
```

Insert 2

```
INSERT INTO events (title, starts, ends, venue_id,
event_id)
VALUES ('April Fools Day', '2018-04-01 00:00:00',
'2018-04-01 23:59:00', NULL, 2);
```

Insert 3: Sin venue (NULL)

```
INSERT INTO events (title, starts, ends, venue_id,
event_id)
VALUES ('Christmas Day', '2018-02-15 19:30:00',
'2018-12-25 23:59:00', NULL, 3);
```

16

```
SELECT e.title, v.name
FROM events e JOIN venues v
ON e.venue_id = v.venue_id;
```

esto solo va a devolver una fila

```
SELECT e.title, v.name
FROM events e LEFT JOIN venues v
ON e.venue_id = v.venue_id;
```

esto al ser outer join, va a mostrar todas las otras filas aunque no coincidan los venue_id

HOMEWORK DIA 1

```
1- SELECT relname, relkind, relpages, reltuples FROM
pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r' AND n.nspname = 'public';
```

gran parte de la información se almacena en el esquema `pg_catalog`.

la base de datos es una base de datos de sí misma. Cuando ejecutas un `CREATE TABLE`, Postgres no solo crea un archivo en disco, sino que hace cientos de inserciones en estas tablas de catálogo.

es decir, posee un gran poder de abstracción la base de datos.

```
2-
SELECT c.country_name
FROM events e
JOIN venues v ON e.venue_id = v.venue_id
JOIN cities ci ON v.country_code = ci.country_code AND
v.postal_code = ci.postal_code
JOIN countries c ON ci.country_code = c.country_code
WHERE e.title = 'Fight Club';
```

```
3-
ALTER TABLE venues
ADD COLUMN active BOOLEAN DEFAULT TRUE;
```

DIA 2

```
1
INSERT INTO events (title, starts, ends, venue_id)
VALUES ('Moby', '2018-02-06 21:00', '2018-02-06 23:00', (
SELECT venue_id
FROM venues
WHERE name = 'Crystal Ballroom'
```



```
)  
);
```

QUERY DENTRO DE QUERY , escrita en lenguaje humano, se inserta el venue_id indirectamente con la subconsulta

```
2  
INSERT INTO events (title, starts, ends, venue_id) VALUES  
( 'Wedding', '2018-02-26 21:00:00', '2018-02-26 23:00:00',  
NULL),  
( 'Dinner with Mom', '2018-02-26 18:00:00', '2018-02-26  
20:30:00', NULL),  
( 'Valentine''s Day', '2018-02-14 00:00:00', '2018-02-14  
23:59:00', NULL);
```

```
3  
SELECT count(title)  
FROM events  
WHERE title LIKE '%Day%';
```

```
4 GROUPING  
SELECT venue_id, count(*)  
FROM events  
GROUP BY venue_id;
```

Esto devuelve las filas en grupos, con una función de COUNT aplicada sobre los mismos.

```
SELECT venue_id  
FROM events  
GROUP BY venue_id  
HAVING count(*) >= 2 AND venue_id IS NOT NULL;
```

Having aplica un filtro sobre el resultado del group.

```
SELECT title, count(*) OVER (PARTITION BY venue_id) FROM  
events;
```

ejemplo de transaccion:

```
INSERT INTO cities (country_code, postal_code, name)
VALUES ('AR', '5505', 'Lujan de Cuyo');
```

```
BEGIN;
```

```
SELECT * FROM cities WHERE postal_code = '5505';
DELETE FROM cities WHERE postal_code = '5505';
SELECT * FROM cities WHERE postal_code = '5505';
```

```
ROLLBACK;
```

```
SELECT * FROM cities WHERE postal_code = '5505';
```

eventos

```
SELECT add_event('House Party', '2018-05-03 23:00',
'2018-05-04 02:00', 'Run''s House', '97206', 'us');
```

Esto devuelve true, ya que el evento por dentro se ocupa de mantener todas las transacciones con exito.

```
CREATE TABLE logs (
event_id integer,
old_title varchar(255),
old_starts timestamp,
old_ends timestamp,
logged_at timestamp DEFAULT current_timestamp
);
```

—

```
CREATE OR REPLACE FUNCTION log_event() RETURNS trigger AS
$$
DECLARE
BEGIN
INSERT INTO logs (event_id, old_title, old_starts,
old_ends)
VALUES (OLD.event_id, OLD.title, OLD.starts, OLD.ends);
RAISE NOTICE 'Someone just changed event #%',
OLD.event_id;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
--creacion de trigger
CREATE TRIGGER log_events
AFTER UPDATE ON events
FOR EACH ROW EXECUTE PROCEDURE log_event();
```

```
UPDATE events
SET ends='2018-05-04 01:00:00'
WHERE title='House Party';
```

```
esto tiene que arrojar:
NOTICE: Someone just changed event #9
```

```
SELECT event_id, old_title, old_ends, logged_at
FROM logs;
```

```
CREATE VIEW holidays AS
SELECT event_id AS holiday_id, title AS name, starts AS
date
FROM events
WHERE title LIKE '%Day%' AND venue_id IS NULL;
```

```
SELECT name, to_char(date, 'Month DD, YYYY') AS date
FROM holidays
WHERE date <= '2018-04-01';
```

holidays es la vista creada arriba

```
ANTES DE ACTIVAR LA CONSULTA
CREATE EXTENSION IF NOT EXISTS tablefunc;
```

```
- por si mismo:
-
SELECT extract(year from starts) as year,
extract(month from starts) as month, count(*)
FROM events
GROUP BY year, month
ORDER BY year, month;
```

```
- si o si
CREATE TEMPORARY TABLE month_count(month INT);
INSERT INTO month_count VALUES (1),(2),(3),(4),(5),
(6),(7),(8),(9),(10),(11),(12);
```

```
SELECT * FROM crosstab(
```

```
'SELECT extract(year from starts) as year,  
extract(month from starts) as month, count(*)  
FROM events  
GROUP BY year, month  
ORDER BY year, month',  
'SELECT * FROM month_count'  
) AS (  
year int,  
jan int, feb int, mar int, apr int, may int, jun int,  
jul int, aug int, sep int, oct int, nov int, dec int  
) ORDER BY YEAR;
```